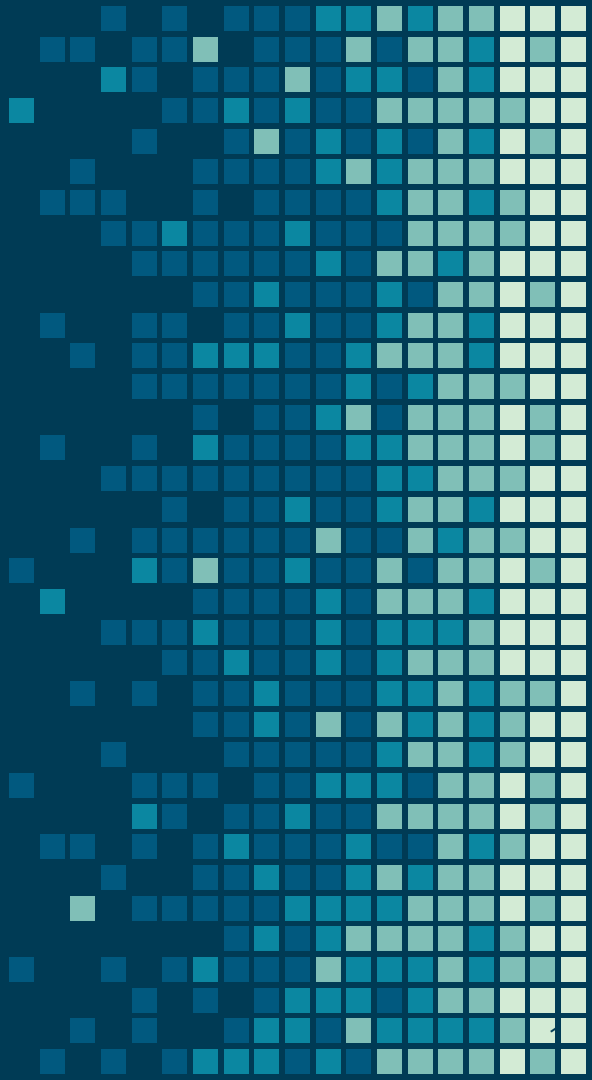
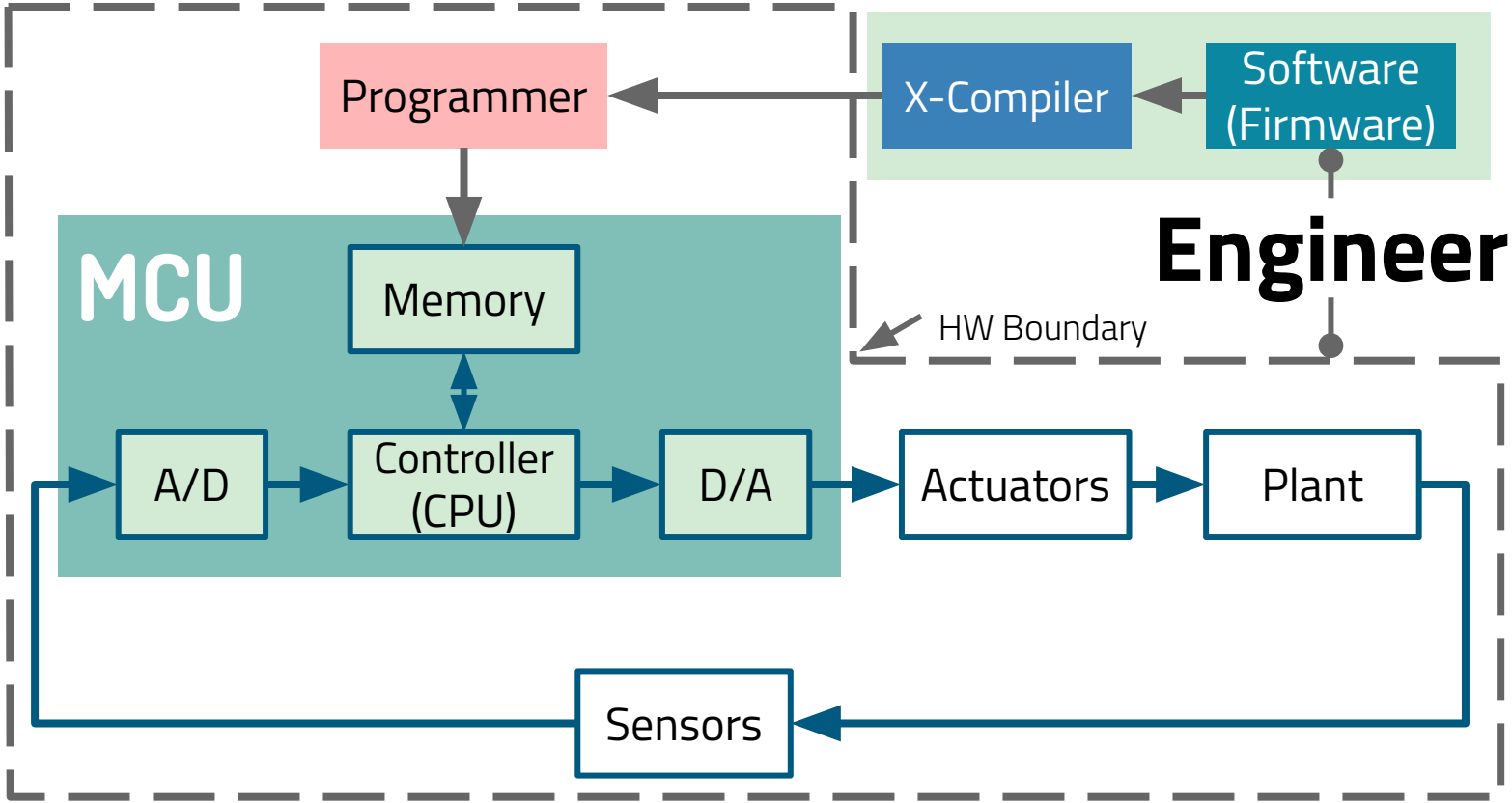
EMBEDDED CONTROL

Prof. Luca De Cicco
luca.decicco@poliba.it
https://c3lab.poliba.it/LDC



Serial Communication Protocols

Back from Lecture #1



Communication

- Making (sub)system communicate is fundamental to be mastered by embedded control systems designers
- Communication can be either digital or analog
- Depends on hardware involved in the communication
- Typically “long distance” communication are digital since it is more immune to medium noise (better if differential!)
- **Digital communication**: data is first quantized then encoded in bits transferred (typically) in the form of a low/high voltage level on a communication medium
- Any communication requires a protocol, i.e., a set of rules to follow for peers to communicate successfully

Communication in ECS

- Communication is typically done “serially” (one bit after another) to decrease wiring needs
- If a large bandwidth is needed parallel communication is used (example: MCU - display communication, i.e. LVDS)
- Serial is commonly used in communication networks mainly due to costs (long distance links need to minimize cabling)
- Serial communication is by all means the most used
- Many communication protocols/technologies are available
- Which one to pick depends on the application

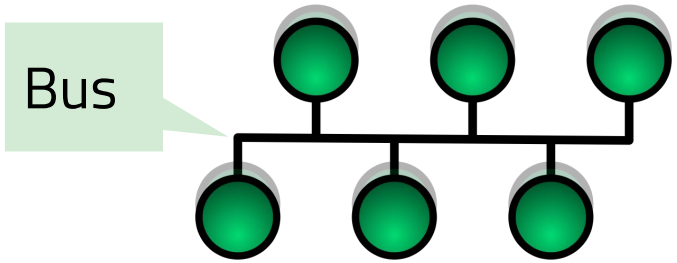
Serial communication Protocols

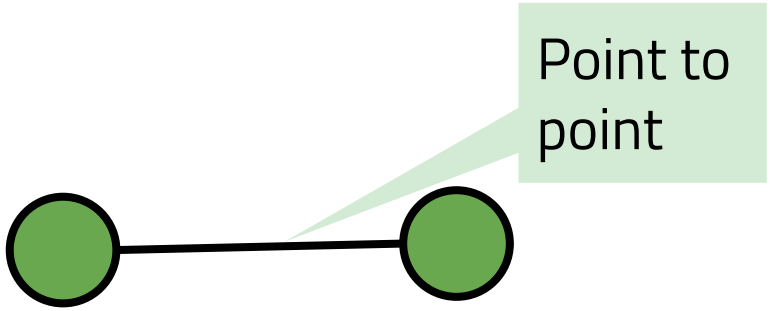
- **Universal Synchronous/Asynchronous Receiver and Transmitter** (USART or UART): 3 (or 2) wires used to connect peer to peer two devices, full-duplex
- **Inter-Integrated Circuit** (I^2C): 2 wires **bus**, supports communication of more than two devices sharing the same bus, half-duplex
- **Serial Peripheral Interface Bus** (SPI): 3 wires bus + 1 chip select per slave, master/slave communication, full-duplex
- **Universal Serial Bus** (USB): 4 wires (up to USB 2.0), half-duplex communication

How to select the communication protocol

- Sometimes you cannot choose
  - Obvious: if the selected sensor only speaks f.i. I2C there is no choice
  - Less obvious: if the MCU is placed far away from the other peer you need a communication technology supporting long distances. A sensor might be good in terms of functionality, but not usable due to communication technology used
- If you can choose, several trade-offs to consider: wiring (the less the better), bandwidth, reliability…

Bus vs point to point





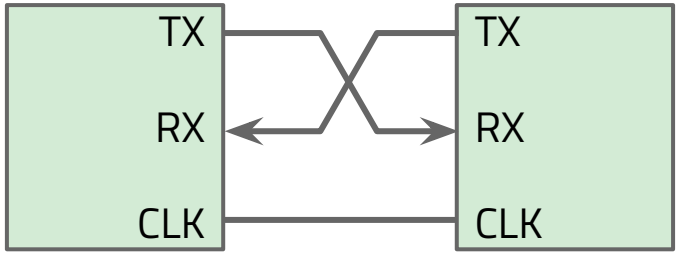
- Bus is a good choice when several devices need to communicate
- Buses are flexible: can add new devices to the bus easily
- Buses share a medium, an arbitration protocol is needed to discipline access (might introduce randomness in latencies)
- Point-to-point (ex. USART) are much simpler but dedicated to a pair of transmitting devices

Universal Synchronous/Asynchronous Receiver and Transmitter (USART)

USART

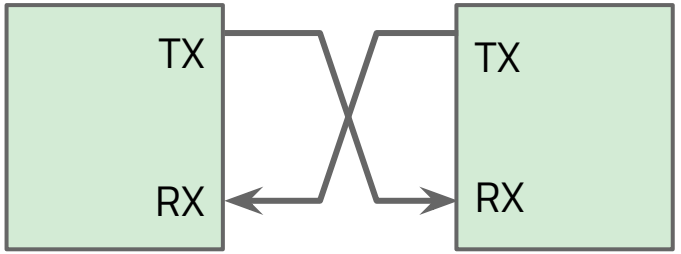
- Widely used due to its simplicity
- Universal: USART is programmable, can be used at different data rates and modes
- Two possible modes:
  - **Synchronous** (USART): a clock is used to synchronize peers
  - **Asynchronous** (UART): no clock is used, peers need to agree on timings (i.e., length of a bit)
- UART/USART can be either:
  - half-duplex: peers can communicate in both directions but not simultaneously
  - full-duplex, both communicate simultaneously

USART vs UART



USART

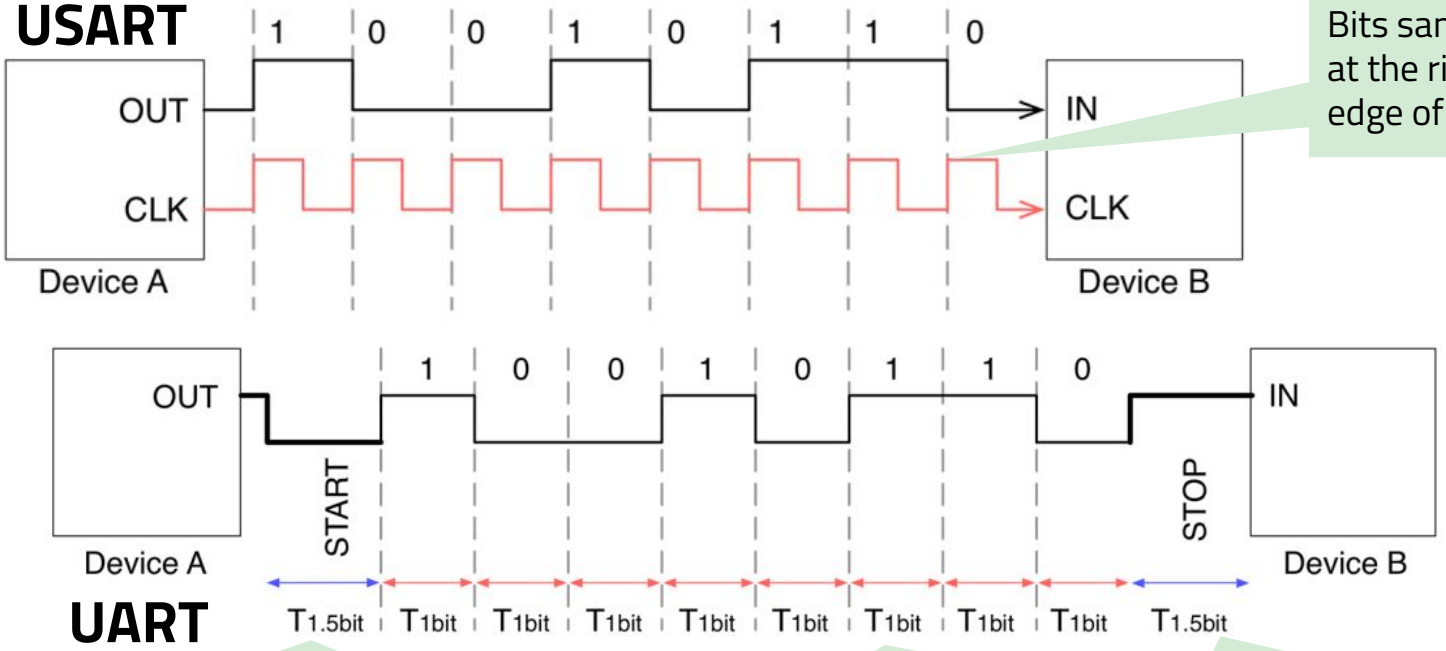
- Bits are transferred synchronously to the common clock (CLK)
- There is no need to specify the length of a bit



UART

- Peers need to agree on the protocol needed to send a byte of information
- Since there is no clock knowing the length of a bit is fundamental

USART vs UART (sending `0b10010110`)



Need to know when a new byte starts. START bit can be longer than 1 bit f.i., $T_{1.5bit}=1.5 \times T_{1bit}$

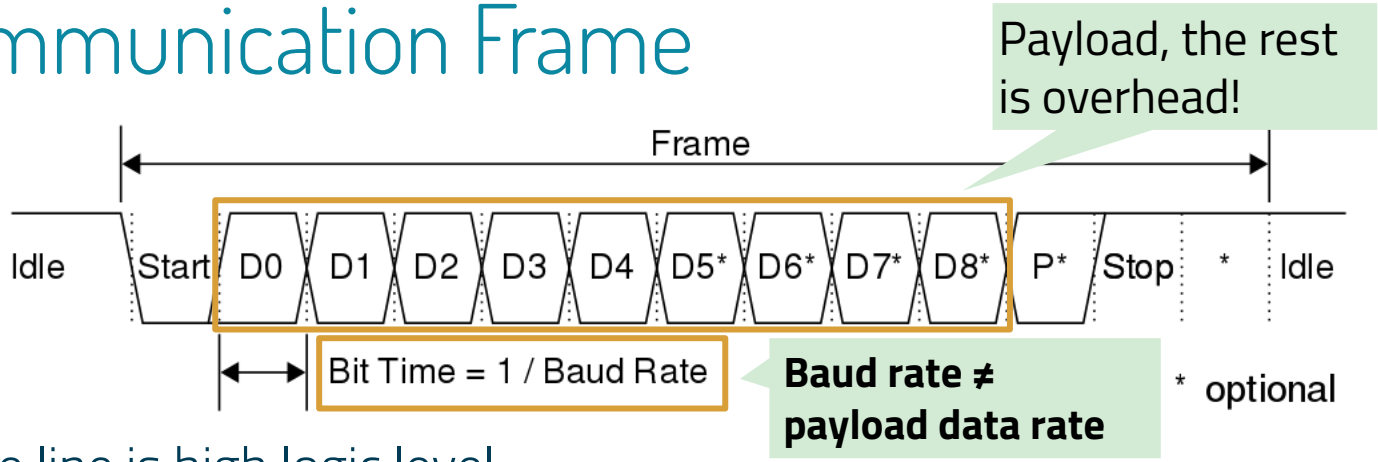
Need to agree on the duration of a bit T_{1bit} to sample bits

Similar to START bit, but for the STOP bit the line is pulled up (high)

Communication Frame

- UART divides data to be transmitted into frames
- A frame is the **smallest complete transmission unit** of data that can be transmitted via UART
- The length of some fields of the frame are configurable
- The duration of each bit depends on the “baud rate”

Communication Frame



- Idle line is high logic level
- The start bit pulls line to LOW (start bit)
- A number of data bits is sent (5-9). Bits can be sent in LSB (least significant bit is sent first) or MSB (opposite)
- An optional parity bit can be added to increase data integrity. Parity can be set to “odd” or “even” or “no parity”
- The frame is closed with a configurable number of stop bits (high logic level)

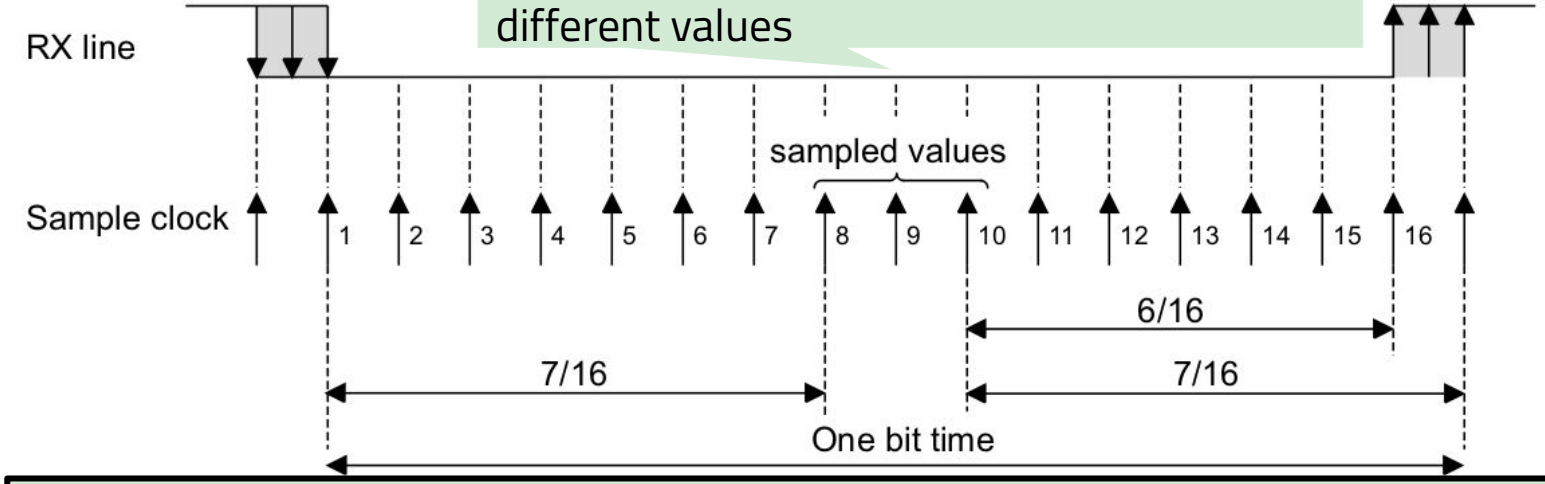
Oversampling

- In order to sample bits and to mitigate sampling errors due to noise and imprecise clocks **oversampling** is used
- In practice for each bit time (1/Baudrate) a number of samples are generated
- Typical values of oversampling are 8 or 16
- This way a logical level is obtained by considering a number of samples in the middle of the bit and using majority voting
- Notice that due to parasitic capacitances square waves are not really square (recall RC circuit)

Example: sampling a bit (16bit oversampl.)

a `0` is sampled

N.B. **Majority voting** is used when sampled different values



baudrate = 115200 (typical), T_{1bit} = 1/baudrate ≅ 8.68 μs

Sampling period: $T_s = T_{1bit}/16$ ≅ 0.5425 μs

UART Clock frequency: $1/T_s$ ≅ 1.84 MHz

How to set the UART baudrate

$$\texttt{Baudrate} = \frac{f_{pCLK}}{8\times(2-\texttt{OVER8})\times\texttt{USARTDIV}}$$

Encoded in the **BRR** register (see manual)

- Baudrate is given (we set it)
- `OVER8` = 1 if we are oversampling by 8 otherwise `OVER8=0` (default) and we are oversampling by 16
- f_{pCLK} is the clock feeding the UART
- Example: baudrate = 9600, f_{pCLK}=80MHz, 16 oversampling gives `USARTDIV`≅520.8333

Example

- Example: baudrate = 9600 f_{pCLK}=80MHz, overs. by 16 gives `USARTDIV`≅520.8333
- USARTDIV is expressed in Q12.4 format (12 bits for mantissa (520) and 4 bits for fraction (0.8333)
- 520 in hex are three nibbles: `0x208`
- The fraction must be expressed as 0.8333 x 2^4=13.333 represented as `0xD`
- Thus: BRR = `0x208D`

25.6.3 Baud rate register (USART_BRR)

Note: The baud counters stop counting if the TE or RE bits are disabled respectively.

Address offset: 0x08

Reset value: 0x0000 0000

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
DIV_Mantissa[11:0]												DIV_Fraction[3:0]			
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw

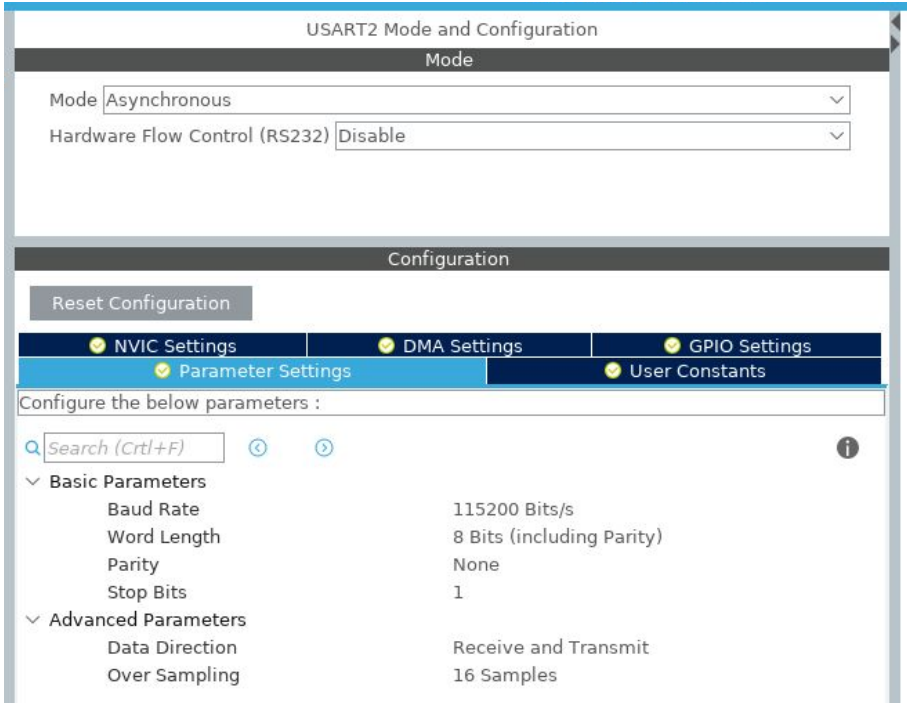
Bits 31:16 Reserved, must be kept at reset value

Bits 15:4 **DIV_Mantissa[11:0]**: mantissa of USARTDIV
These 12 bits define the mantissa of the USART Divider (USARTDIV)

Bits 3:0 **DIV_Fraction[3:0]**: fraction of USARTDIV
These 4 bits define the fraction of the USART Divider (USARTDIV). When OVER8=1, the DIV_Fraction3 bit is not considered and must be kept cleared.

Configuring the U(S)ART with CubeMX

- CubeMX provides a convenient way to set the USART
- Activate the UART from "connectivity" panel (left)
- Configure it through the configuration panel
- **Notice** word length includes the parity bit!
- Oversampling by 16 by default and more roubs
- Use 8bit for higher baud rates



Using the USART with HAL

- Once the USART is configured, CubeMX will generate the USART driver and configure for us the peripheral
- Recall to select “Generate peripheral initialization as a pair of .c/.h files per peripheral” in “Code Generator”)
- **`Src/usart.c`** contains the peripheral initialization code
- `Drivers/STM32F4xx_HAL_DRIVER/Src/stm32f4xx_hal_uart.c` contains the HAL UART driver
- In `usart.c` the **peripheral handle** `huartX` is generated
- This is a common approach used by HAL to handle every peripheral

Typical values for **Baudrate**

Baud rate		Oversampling by 16		Oversampling by 8	
S.No	Desired (Bps)	Actual	%Error	Actual	%Error
2	2400	2400	0	2400	0
3	9600	9600	0	9600	0
4	19200	19200	0	19200	0
5	38400	38400	0	38400	0
6	57600	57620	0.03	57590	0.02
7	115200	115110	0.08	115250	0.04
8	230400	230760	0.16	230210	0.8
9	460800	461540	0.16	461540	0.16
10	921600	923070	0.16	923070	0.16
11	2000000	2000000	0	2000000	0
12	3000000	3000000	0	3000000	0
13	4000000	N.A.	N.A.	4000000	0
14	5000000	N.A.	N.A.	5052630	1.05
15	6000000	N.A.	N.A.	6000000	0

- Example: USART clock is set to 48MHz
- Grey cells show cases where BRR quantization causes baud-rate error
- Check tables 147-156 for different clock freqs
- The larger the error%, the larger the probability to incur in sampling issues

Baud rate example (115200)



Baudrate=115200

Theoretical T1bit= 1/Baudrate=**8.68us**

Measured Time over 8-bit interval = 69.5 us

Measured T1bit = 69.5 us/8=**8.68us**

UART handle typedef at a glance

```
typedef struct __UART_HandleTypeDef {
  USART_TypeDef      *Instance;        /* UART registers base address        */
  UART_InitTypeDef   Init;             /* UART communication parameters      */
  uint8_t            *pTxBuffPtr;      /* Pointer to UART Tx transfer Buffer */
  uint16_t           TxXferSize;       /* UART Tx Transfer size              */
  __IO uint16_t      TxXferCount;      /* UART Tx Transfer Counter           */
  uint8_t            *pRxBuffPtr;      /* Pointer to UART Rx transfer Buffer */
  uint16_t           RxXferSize;       /* UART Rx Transfer size              */
  __IO uint16_t      RxXferCount;      /* UART Rx Transfer Counter           */
  DMA_HandleTypeDef  *hdmatx;          /* UART Tx DMA Handle parameters      */
  DMA_HandleTypeDef  *hdmarx;          /* UART Rx DMA Handle parameters      */
  HAL_LockTypeDef    Lock;             /* Locking object                     */
  __IO HAL_UART_StateTypeDef gState;   /* UART global state  */
  __IO HAL_UART_StateTypeDef RxState;  /*!< UART Rx operations state */

  __IO uint32_t      ErrorCode;        /*!< UART Error code                   */
} UART_HandleTypeDef;
```

Important Fields (1/2)

- `Instance`: pointer to USART registers used to access peripheral. Example: `USART2` (defined in `stm32f446xx.h`)
- `Init`: configuration parameters used in the peripheral initialization phase (see `usart.c`). Struct containing uint32_t fields:
  - `BaudRate, WordLength, StopBits, Parity, Mode, HwFlowCtl, OverSampling`
  - For each of these fields, `#define`'s are provided

Important fields (2/2)

- `pTxBuffPtr/pRxBuffPtr`: pointers to TX and RX buffers. Such pointers are provided by the programmer
  - `pTxBufferPtr` is used as a source for the UART
  - `pRxBufferPtr` is used to store data received by UART
- TxXferSize/RxXferSize: amount of data to be transmitted/received
- `TxXferCount/RxXferCount`: used to keep track of the amount of bytes transmitted/received
- `Lock`: field used by HAL to lock concurrent access to UART
- General lock/unlock approach used by HAL for a peripheral : check **`__HAL_LOCK(__HANDLE__)`**

HAL API to access UART in polling mode

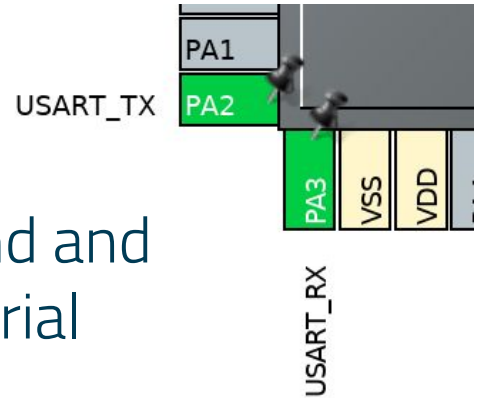
```
HAL_StatusTypeDef HAL_UART_Receive(UART_HandleTypeDef *huart,
uint8_t *pData, uint16_t Size, uint32_t Timeout)
```

```
HAL_StatusTypeDef HAL_UART_Transmit(UART_HandleTypeDef *huart,
uint8_t *pData, uint16_t Size, uint32_t Timeout)
```

- `huart`: pointer to *handle* generated by CubeMX. Notice that handles are not generated as pointers (need to use & operator)
- `pData`: pointer to the area of memory used to store data when transferring, or to get data to transfer (`pTxBuffPtr/pRxBuffPtr`)
- `Size`: size of data to be transferred/read from UART (`TxXferSize/RxXferSize`)
- `Timeout`: maximum amount of milliseconds to wait before considering that an operation has timed out

How to use the Nucleo UART over USB

- NUCLEO boards have a UART peripheral physically connected to the USB port used for flashing firmwares (serial over USB)
- In the NUCLEO-F446XX case USART2 is the peripheral connected to USB through pins `PA2` (TX) and `PA3` (RX)
- When connecting the NUCLEO to the USB a device is created (Windows `COMX`, Linux `/dev/ttyACM0`)
- A serial console has to be installed to send and receive data over the serial (google for serial terminal and pick one)



Example: receive data

- Goal: write a firmware to receive data sent from the serial terminal installed on your PC
- Set a timeout equal to 100 ms
- When receiving a character toggle the LD2 led
- Notice: the serial needs to be correctly set on the PC host to match the configuration given in the firmware

HAL API to access UART in interrupt mode

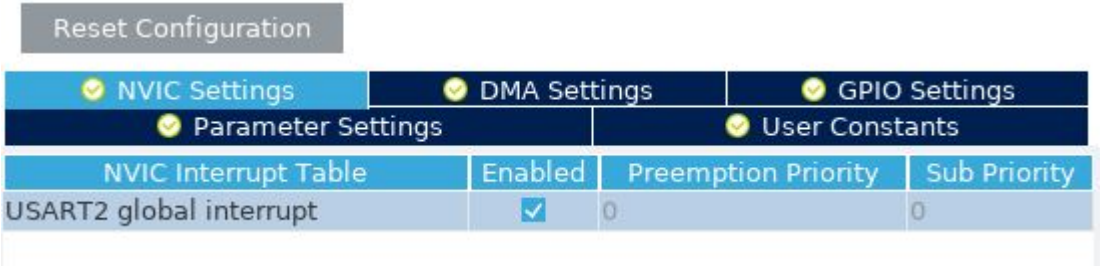
```
HAL_StatusTypeDef HAL_UART_Receive_IT(UART_HandleTypeDef *huart,
uint8_t *pData, uint16_t Size)
```

```
HAL_StatusTypeDef HAL_UART_Transmit_IT(UART_HandleTypeDef
*huart, uint8_t *pData, uint16_t Size)
```

- Exactly the same of polling-mode but there is no timeout
- `huart`: pointer to *handle* generated by CubeMX. Notice that handles are not generated as pointers (need to use & operator)
- `pData`: pointer to the area of memory used 1) to store data when transferring, or to get data to transfer (`pTxBuffPtr/pRxBuffPtr`)
- `Size`: size of data to be transferred/read from UART (`TxXferSize/RxXferSize`)

CubeMX configuration

- In order to enable interrupt mode, the corresponding IRQ must be enabled
- Select the correct USART (USART2) and click on “NVIC settings”
- Enable “**USART2 global interrupt**”



Interrupt related generated code

In `usart.c` interrupts are enabled:

```
HAL_NVIC_SetPriority(USART2_IRQn, 0, 0);
HAL_NVIC_EnableIRQ(USART2_IRQn);
```

In `stm32f4xx_it.c` we have the handler of the global UART IRQ:

```
void USART2_IRQHandler(void) {
  HAL_UART_IRQHandler(&huart2);
}
```

How to get notified

- Different callbacks are called in the handler USARTx_IRQHandler depending on the IRQ fired
- Must be implemented in the firmware (**recall**: default empty callback are always defined as `weak` functions in the drivers)
- `HAL_UART_TxCpltCallback`: called when the transmission has been completed (all bytes have been transferred)
- `HAL_UART_TxHalfCpltCallback`: called when half of the bytes have been transmitted (active only in DMA mode!)
- `HAL_UART_RxCpltCallback`: called when the reception of data has been completed (all bytes have been received)
- `HAL_UART_RxHalfCpltCallback`: called when half of the bytes have been received (active only in DMA mode!)

Lab experience: polling mode

- Write a firmware that reads **in polling mode** characters from the UART
- Recall, you need to connect to the UART through a serial terminal
- Put the stored characters in a buffer with a fixed length equal to 16 characters
- Upon reading 16 characters send the 16 characters back on the serial and restart
- When the 16 characters have been sent back toggle the LED LD2

Lab experience: interrupt mode

- Write a firmware that reads **in interrupt mode** characters from the UART
- Put the stored characters in a buffer with a fixed length equal to 16 characters
- When reading half of the characters (i.e., 8 chars) send to the other peer the message “received half chars”
- Upon reading 16 characters send the 16 characters back on the serial
- When the 16 characters have been sent back toggle the LED LD2